

中山大學

SUN YAT-SEN UNIVERSITY

实验 0x04 实验报告

用户程序与系统调用

姓　　名：　何亮

学　　号：　24325005

教学班号：　202520804（陈鹏飞老师班级）

专　　业：　计算机科学与技术

院　　系：　计算机学院

2025 学年第二学期

1. 用户程序

1.1. 编译用户程序

根据文档配置好用户程序的 `Cargo.toml` 之后，即可以正常使用 `cargo build` 进行编译。

```
(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab/crates/app/hello$ cargo build -Zjson-target
-spec
   Compiling ysos_syscall v0.4.0 (/home/linkzelda/OS-lab/crates/syscall)
   Compiling yslib v0.4.0 (/home/linkzelda/OS-lab/crates/lib)
   Compiling ysos_hello v0.4.0 (/home/linkzelda/OS-lab/crates/app/hello)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.52s
(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab/crates/app/hello$
```

1.2. 加载程序文件

根据文档定义 `App` 结构体，使用 `const` 指定用户程序数组最大长度，同时定义 `AppListRef` 类型存储 `AppList` 的引用，同时把这些类型的声明周期都改为 `'static` 便于代码编写（由于它是在 `boot` 时期启用到内核程序终止位置，用 `static` 符合其预期的生命周期）。

随后，我们在 `fs.rs` 中，将所有 `APP/` 下的文件转换成 `ELF` 文件类型，并存入 `AppList` 中，交给 `boot` 去处理。然后我们对应修改一下其他部分的代码，让 `AppList` 中的信息可以正确传递给内核，并在调用 `list_app` 函数的时候可以正常展示。

目前我的 `list_app` 函数可以展示各个 `app` 的文件名，占用空间大小以及程序入口点的地址，如下

```
[INFO] ysos_kernel::proc - Process Manager Initialized.
[INFO] ysos_kernel - Interrupts Enabled.
[INFO] ysos_kernel - YatSenOS initialized.
[+] App list:
  App Name        | Size       | Entry Point
  ----------------+------------+--------------------
  hello           |    34320 B | 0x0000111100001020
  factorial       |    34872 B | 0x0000111100001490
  sh              |    35160 B | 0x0000111100001550
```

1.3. 生成用户程序

这部分我们需要实现 `ProcessManager` 中的 `spawn` 函数，如下

```rust
// FIXED: load elf to process pagetable
let text_pages = inner.load_elf(elf);
let stack_pages = stack::STACK_DEF_PAGE as usize;
let total_memory_bytes = (text_pages + stack_pages) * PAGE_SIZE as usize;
inner.set_env("ELF_PAGES", &alloc::format!("{}", text_pages));
inner.set_env("MEM_USAGE", &alloc::format!("{}", total_memory_bytes));

// FIXED: alloc new stack for process
let pid = proc.pid();
let stack_top = inner.vm_mut().init_proc_stack(pid);
let entry_point = VirtAddr::new(elf.header.pt2.entry_point() as usize as u64);
inner.init_stack_frame(entry_point, stack_top);

// FIXED: mark process as ready
inner.pause();

// FIXED: something like kernel thread
self.add_proc(pid, proc.clone());
self.push_ready(pid);
```

具体而言，我们需要先把 `ELF` 文件加载到内核的页表，并设置相关的信息，便于后续进行 `app` 信息的输出；然后为该进程设置对应的栈，并放入就绪队列，设置进程状态为就绪态。

然后对应修改一下 `load_elf` 函数，添加 `user_access` 标志位，保护内核态内存不会被用户态程序访问。这部分代码请见各个 `load_elf` 和 `load_segment` 函数的部分。

随后，为 Ring 3 的代码段和数据段添加对应的选择子，并在初始化栈帧的时候，将其选择子设置为 Ring3 特权级，这部分代码请见 `gdt.rs` 和 `context.rs` 。

2. 系统调用的实现

这部分文档详细介绍了本实验系统调用的调用约定，包括系统调用号，寄存器的定义和设定等。

我们需要在系统调用的 `mod.rs` 中补全中断注册函数, 即为其设置对应的中断处理函数, 以及对应的栈和特权级

```rust
// FIXED: register syscall handler to IDT
//        - standalone syscall stack
let handler = idt[consts::Interrupts::Syscall as u8]
    .set_handler_fn(syscall_handler)
    .set_stack_index(gdt::SYSCALL_IST_INDEX);
//        - ring 3
handler.set_privilege_level(x86_64::PrivilegeLevel::Ring3);
```

3. 用户态库的实现

3.1. 动态内存分配

我们需要支持用户态堆的动态内存，代码如下

```
// FIXED: use elf::map_range to allocate & map
//        frames (R/W/User Access)
let range_start =
Page::<Size4KiB>::containing_address(VirtAddr::new(USER_HEAP_START as u64));
let range_end = range_start + USER_HEAP_PAGE as u64;

let flags = PageTableFlags::PRESENT
    | PageTableFlags::WRITABLE
    | PageTableFlags::USER_ACCESSIBLE
    | PageTableFlags::NO_EXECUTE;

for page in Page::range(range_start, range_end) {
    let frame = frame_allocator
        .allocate_frame()
        .ok_or(MapToError::FrameAllocationFailed)?;
    unsafe {
        mapper.map_to(page, frame, flags, frame_allocator)?.flush();
    }
}
```

具体而言，首先我们计算需要分配的页表范围，添加对应的标志位，让用户态程序可以访问，然后逐页进行内存分配和建立映射。

3.2. 标准输入输出

我们此处需要实现 `sys_wtite` 函数，用来处理 `write` 系统调用

```
// FIXED: get buffer and fd by args
//       - core::slice::from_raw_parts
let buf = unsafe { core::slice::from_raw_parts(ptr, len) };
// FIXED: call proc::write -> isize
let ret = crate::proc::write(fd, buf);
// FIXED: return the result as usize
if ret < 0 { 0 } else { ret as usize }
```

即从系统调用传递的参数中获取要写入的字节数，缓冲区的起始地址等，然后将用户缓冲区转化为 `&[u8]`，便于调用 `proc::write` 函数，并处理和返回其返回值，然后在分发函数中将该返回值放入 `rax` 寄存器

```
Syscall::Write => {
    context.set_rax(sys_write(&args));
}
```

`sys_read` 函数的实现类似，这里需要把用户缓冲区转化为 `&mut [u8]`，使用 `from_raw_parts_mut` 函数，其他部分这里不再赘述。

3.3. 进程的退出

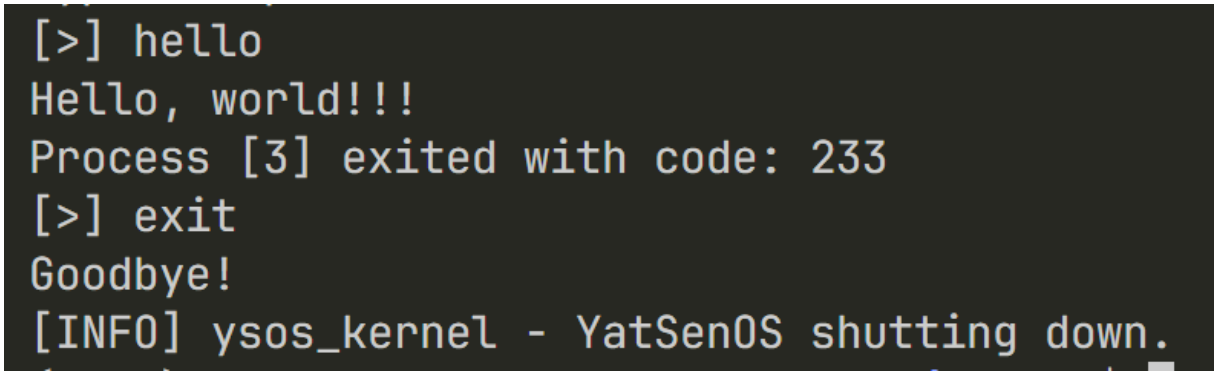
这部分需实现 `exit` 函数，具体而言，就是调用 `manager.kill_current` 函数，然后一步步将 `kill` 函数委托下去处理，最后释放资源

```
// FIXED: set exit code
self.exit_code = Some(ret);

// FIXED: set status to dead
self.status = ProgramStatus::Dead;

// FIXED: take and drop unused resources
self.proc_data.take();
self.proc_vm.take();
```

此后，操作系统可以正常运行 `hello` 程序并退出！



3.4. 进程的创建与等待

我们需要实现 `still_alive` 函数

```
// check if the process is still alive
if let Some(proc) = get_process_manager().get_proc(&pid) {
    return proc.read().status() != ProgramStatus::Dead;
} else {
    false
}
```

`still_alive` 函数根据 `pid` 获取进程状态进行判断即可，注意这里需要禁止中断，防止时钟中断改变进程状态，导致函数返回值错误，或者也有可能由于这里需要获取锁而造成死锁。

然后，我们需要实现其他系统调用，并写入分发函数中，代码请见 `interrupt/syscall/mod.rs` 与 `interrupt/syscall/service.rs`。

此后，我们的 OS 可以正常支持用户态的系统调用，以及用户态程序的正常运行了！

4. 测试程序

我目前实现的 `shell` 支持如下命令

```
(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab$ make run
[INFO] ysos_kernel::interrupt - Interrupts Initialized.
[INFO] ysos_kernel::proc - Process Manager Initialized.
[INFO] ysos_kernel - Interrupts Enabled.
[INFO] ysos_kernel - YatSenOS initialized.
[+] App list:
  App Name        | Size       | Entry Point
  ----------------+------------+------------------
  hello           |    34320 B | 0x0000111100001020
  factorial       |    34872 B | 0x0000111100001490
  sh              |    35160 B | 0x0000111100001550

Welcome to YatSenOS Shell!
Type 'help' for instructions.
[>] help
YatSenOS Shell Help
Commands:
  list_app    - List available applications
  stat        - List running processes
  clear       - Clear the screen
  help        - Show this help message
  <app_name> - Run the specified application
Student ID: 24325005
```

可以运行给出的 `factorial` 测试性能

```
[>] factorial
Input n: 999999
  PID | PPID | Process Name |  Ticks  | Status | Memory Usage
 #  2 | #  1 | sh           |  220011 | Ready | 24576 B (6 Pages)
 #  1 | #  0 | kernel       |  220025 | Ready | 2097152 B (512 Pages)
 #  3 | #  2 | factorial    |    6354 | Running | 16023552 B (3912 Pages)
Queue  : [2, 1]
CPUs   : [0: 3]
The factorial of 999999 under modulo 1000000007 is 128233642.
Process [3] exited with code: 0
```

占用的页面为 `3912`，内存约为 `15.3MiB`，结果输出正确，符合预期!

5. 思考题

5.1. 是否可以在内核线程中使用系统调用? 并借此来实现同样的进程退出能力? 分析并尝试回答。

系统调用指的是: 运行在用户空间的程序向操作系统内核请求需要更高权限运行的服务。系统调用是给特权级为 ring3 的代码来执行的。ring0 特权级的内核进程直接使用普通函数调用的方式来实现对应的功能。

理论上来说，我们是可以通过一些巧妙的实现，如不屏蔽 Ring0 到 Ring0 的切换等，在内核线程中使用系统调用，因为我们为系统调用设置了独立的栈，和内核自己的栈是不会冲突，可以正常使用。

我们可以尝试在 `main.rs` 中通过如下汇编触发 `sys_exit` 系统调用，其中 `rax` 传入的是系统调用号（`sys_exit` 对应的是 `60`），`rdi` 传入的是 `exit` 需要的参数

```
unsafe {
    core::arch::asm!(
        "int 0x80",
        in("rax") 60,
        in("rdi") 0,
    );
}
```

我们 `make_run` 一下会发现会触发如下异常

```
[>] exit
Goodbye!
[ERROR] ysos_kernel::utils::macros -

ERROR: panicked at crates/kernel/src/proc/process.rs:141:31

called `Option::unwrap()` on a `None` value

Stack trace (most recent call last):
  #00: 0xffffff000001e4fb
  #01: 0xffffff000001e4c1
  #02: 0xffffff000001ecf8
  #03: 0xffffff0000009715
  #04: 0xffffff0000018916
  #05: 0xffffff0000017e28
  #06: 0xffffff0000008059
  #07: 0x00000000043bf417
```

我们仔细分析，发现这个异常是由于时钟中断来临的时候，进程调度器使用的 `switch_next` 无法访问页表，这说明我们的 `sys_exit` 调用成功了，真的清空并回收了内核进程的页表资源。因此我们也就证明了在内核线程中触发系统调用的可行性。

5.2. 为什么需要克隆内核页表？在系统调用的内核态下使用的是哪一张页表？用户态程序尝试访问内核空间会被正确拦截吗？尝试验证你的实现是否正确。

因为在用户进程中，会需要用虚拟地址来访问内核的代码和数据（如发生中断或者系统调用等），因此我们需要克隆内核的页表来保证上述虚拟地址可以正确地访问。

在系统调用的内核态下，使用的是当前触发该系统调用的这个用户进程自己的页表。我们可以通过在系统调用的内核函数处理函数中输出用户进程的 `CR3` 寄存器值和当前 `CR3` 寄存器的值，可以发现两者是一样的

```
[INFO] ysos_kernel::interrupt::syscall - current cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - process cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - current cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - process cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - current cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - process cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - current cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - process cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - current cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - process cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - current cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - process cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - current cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - process cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - current cr3: 0x164000
```

尝试在 `hello` 中加入访问内核态起始地址的代码，

```
let kernel_ptr = 0xffffff0000000000u64 as *const i32;
let secret = unsafe { *kernel_ptr };

println!("The secret number is: {}", secret);
```

然后尝试运行，如下图，可以看到 `manager` 捕获到了这个非法访问，说明我们正确拦截了用户程序对内核空间的访问

```
[>] hello
Hello, world!!!
[WARN] ysos_kernel::proc::manager - Page fault at 0xffffff0000000000 wit
h protection violation.
[WARN] ysos_kernel::interrupt::exceptions - EXCEPTION: PAGE FAULT, ERROR
_CODE: PageFaultErrorCode(PROTECTION_VIOLATION | USER_MODE)

Trying to access: 0xffffff0000000000
Caused by PID: 3
InterruptStackFrame {
    instruction_pointer: VirtAddr(
        0x11110000101f,
    ),
    code_segment: SegmentSelector {
        index: 5,
        rpl: Ring3,
    },
    cpu_flags: RFlags(
```

5.3. 为什么在使用 `still_alive` 函数判断进程是否存活时，需要关闭中断？在不关闭中断的情况下，会有什么问题？

因为 `still_alive` 函数需要调用 `get_process_manager()` 来访问某个进程的状态，此时需要获得 `ProcessManager` 的自旋锁。如果我们不关闭中断，此时会由于时钟中断来临，时钟中断内部会进行进程的调度，也需要获得 `ProcessManager` 的锁，从而一直自旋等待，造成死锁；其次，如果时间恰好不会造成死锁，正确处理了进程调度，也可能会在其他中断事件发生的过程中修改了某些进程的状态，让一个进程的状态被 `kill` 掉，那么这个时候 `still_alive` 的返回值就可能会和实际不一样，造成错误。

5.4. 对于如下程序，使用 `gcc` 直接编译：

```c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

从本次实验及先前实验的所学内容出发，结合进程的创建、链接、执行、退出的生命周期，参考系统调用的调用过程（可以仅以 Linux 为例），解释程序的运行。

首先我们使用 `gcc` 编译，编译器通过预处理，编译，汇编，链接等将源代码与标准库代码等链接在一起，生成一个 ELF 文件。

然后在 Linux 中执行的时候，内核会通过 `fork()` 系统调用生成一个子进程，然后克隆内核的页表，分配 PID 等来创建这个进程。随后，加载并解析前面生成的 ELF 文件，建立页表映射和用户栈。最后从系统调用返回会用户态，进入程序入口点开始执行。

该程序使用了 `printf` 函数，这个函数的参数会现在用户态进行初步的处理，然后触发 `write` 系统调用，用 `rdi` 等寄存器传参，随后进入 Ring0 特权级的内核态开始处理串口输出。

在输出了 `Hello World!` 后，系统调用返回 Ring3 特权级的用户态，最后 `return 0` 会调用 `exit` 系统调用（过程和上述类似，这里不再赘述，这里传入的参数为 `0`）。然后，该进程变为僵尸进程，内核通过调用 `wait` 来回收资源。

5.5. `x86_64::instructions::hlt` 做了什么？为什么这样使用？为什么不可以在用户态中的 `wait_pid` 实现中使用？

`x86_64::instructions::hlt` 用于把当前指令执行的过程挂起，让出 CPU，直到下一次的外部中断来临，然后继续执行下面的代码。我们在 `wait()` 函数中，我们需要的是等待线程结束然后返回。如果我们一直使用循环来进行忙等待，就会一直占用 CPU，导致资源浪费，使用这个函数可以让出 CPU 供其他线程使用，提升 CPU 利用率，然后在时钟中断来临的时候可以继续检查线程是否 `Dead`，保证原功能正常执行的同时提升性能。

由于 `x86_64::instructions::hlt` 是一条 Ring0 特权级指令，只能在内核态下执行，因此不可以在用户态 Ring3 特权级下的 `wait_pid` 中使用。

5.6. 只有当进行用户输入（触发了串口输入中断）的时候，会触发奇怪的 Page Fault，然而进程切换、内存分配甚至 fork 等系统调用都很正常。请查阅资料，了解特权级栈的作用，实验说明这一系列中断的触发过程，尝试解释这个现象。

- 可以使用 intdbg 参数，或 ysos.py -i 进行数据捕获。
- 留意 0x0e 缺页异常和缺页之前的中断的信息。
- 注意到一个不应当存在的地址……？

我们注释掉特权级栈的设置代码，然后 `make run` 输入，观察到会出现如下错误

```
Welcome to YatSenOS Shell!
Type 'help' for instructions.
[>] [WARN] ysos_kernel::interrupt::exceptions - EXCEPTION: PAGE FAULT, E
RROR_CODE: PageFaultErrorCode(CAUSED_BY_WRITE)

Trying to access: 0xfffffffffffffff8
Caused by PID: 2
InterruptStackFrame {
    instruction_pointer: VirtAddr(
        0x111100001910,
    ),
    code_segment: SegmentSelector {
        index: 5,
        rpl: Ring3,
    },
    cpu_flags: RFlags(
        IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG | ZERO_FLAG | PARITY_FLAG
| 0x2,
    ),
    stack_pointer: VirtAddr(
        0x3ffeffffff00,
    ),
    stack_segment: SegmentSelector {
        index: 6,
        rpl: Ring3,
    },
}
[ERROR] ysos_kernel::utils::macros -

ERROR: panicked at crates/kernel/src/interrupt/exceptions.rs:131:9

Cannot handle page fault!
```

引发了缺页异常，注意到是由于尝试写入数据到一个 `0xfffffffffffffff8` 地址。这是因为，用户输入时会发生串口输入中断，此时会把当前上下文压栈，由于未设置栈，因此栈顶指针的值为 `0x0`，当插入一个元素后发生了下溢，从而导致访问了 `0xfffffffffffffff8` 造成了无法处理的缺页异常。

对于进程切换、内存分配甚至 fork 等系统调用，因为进程切换时基于时钟中断的，内存分配是基于缺页中断的，这两种中断和系统调用我们都为其分配了独立的栈，因此不会触发缺页异常。

6. 加分项

6.1. 尝试在 `ProcessData` 中记录代码段的占用情况，并统计当前进程所占用的页面数量，并在打印进程信息时，将进程的内存占用打印出来。

我们可以在 `Process` 的 `Display` 实现中加入占用内存的显示，我们在 `spawn` 一个进程的时候存储了代码段的内存占用情况，在这里我们可以通过占用的虚存和代码段占用的页数情况算出总的占用的页数和内存

```
let total_page = elf_pages
    + inner.vm().memory_usage() / crate::memory::PAGE_SIZE;
let current_mem_bytes = elf_pages * crate::memory::PAGE_SIZE
    + inner.vm().memory_usage();
```

目前的效果如下

```
 PID | PPID | Process Name |  Ticks   | Status | Memory Usage
#  2 | #  1 | sh           |   35153 | Ready | 24576 B (6 Pages)
#  1 | #  0 | kernel       |   35165 | Ready | 2097152 B (512 Pages)
#  3 | #  2 | factorial    |    7088 | Running | 16023552 B (3912 Pages)
```

6.2. 尝试在 `kernel/src/memory/frames.rs` 中实现帧分配器的回收功能 `FrameDeallocator`，作为一个最小化的实现，你可以在 `Allocator` 使用一个 Vec 存储被释放的页面，并在分配时从中取出。

我们在 `BootInfoFrameAllocator` 加入了一个 `freed_frames: Vec<PhysFrame>` 成员，保存释放的页面，并在分配页面的时候优先在这里面取页面，如果没有释放的页面，才申请一个新的物理页面

```
// FIXED: 先看看内存回收池里面有没有限制的页面
if let Some(reused_frame) = self.freed_frames.pop() {
    self.used += 1;
    return Some(reused_frame);
}
```

6.3. 基于帧回收器的实现，在 `elf` 中实现 `unmap_range` 函数，从页表中取消映射一段连续的页面，并使用帧回收器进行回收。之后，在合适的地方，结合 `ProcessData` 中存储的页面信息，利用这个函数实现进程栈的回收。其他进程资源（如页表、代码段、数据段等）的回收将会在后续实验中实现，目前暂时不需要考虑。

利用加分项 `2` 中实现的帧回收器实现 `unmap_range` 函数，如下

```rust
pub fn unmap_range(
    addr: u64,
    count: u64,
    page_table: &mut impl Mapper<Size4KiB>,
    frame_deallocator: &mut impl FrameDeallocator<Size4KiB>,
) -> Result<(), UnmapError> {
    let range_start = Page::containing_address(VirtAddr::new(addr));
    let range_end = range_start + count;

    trace!(
        "Unmap Page Range: {:?}({})",
        Page::range(range_start, range_end),
        count,
    );

    // 逐个页表取消映射，并交给帧回收器回收
    for page in Page::range(range_start, range_end) {
        if let Ok((phys_frame, flusher)) = page_table.unmap(page) {
            flusher.flush();

            unsafe {
                frame_deallocator.deallocate_frame(phys_frame);
            }
        } else {
            warn!("Failed to unmap page: {:#x}", page.start_address().as_u64());
        }
    }

    Ok(())
}
```

然后我们就可以利用这个函数实现栈内存的回收，算出需要释放的栈的起始地址和页面数，传入 `unmap_range` 函数即可

```
impl Drop for ProcessVm {
    fn drop(&mut self) {
        let mapper = &mut self.page_table.mapper();
        let alloc = &mut *get_frame_alloc_for_sure();

        let stack_start_addr = self.stack.range.start.start_address().as_u64();

        let stack_page_count = self.stack.usage;

        if stack_page_count > 0 && stack_start_addr != 0 {
            trace!("Deallocating Process Stack at {:#x}", stack_start_addr);
            let _ = unmap_range(stack_start_addr, stack_page_count, mapper,
alloc);
        }

        // TODO
    }
}
```

6.4. 尝试利用 `uefi::runtime::get_time()` 和 `chrono` crate，获取当前时间，并将其暴露给用户态，以实现 `sleep` 函数。

我们首先在 `service.rs` 中使用 `uefi::runtime::get_time()` 实现 `sys_get_time`

```
pub fn sys_get_time() -> usize {
    let t = uefi::runtime::get_time().expect("Failed to get UEFI time");

    let time_ms = t.second() as usize * 1000
        + t.minute() as usize * 60 * 1000
        + t.hour() as usize * 3600 * 1000
        + (t.nanosecond() / 1_000_000) as usize;

    time_ms
}
```

定义 `GetTime` 的系统调用号为 `65530`，并在分发函数里面进行分发，然后再在 `lib` 里面封装该系统调用，供用户态使用 `sys_time` 系统调用

```
#[inline(always)]
pub fn sys_time() -> usize {
    syscall!(Syscall::GetTime)
}
```

我们可以在 `app/` 中写一个 `sleep` 用户程序来测试，由于使用题目中的 `chrono` 会与上面的 `#![no_std]` 冲突，这里使用了简化的实现

```rust
#![no_std]
#![no_main]

use lib::*;
extern crate lib;

pub fn sleep(millisecs: usize) {
    let start = sys_time();
    let dur = millisecs;

    loop {
        let current = sys_time();
        if (current - start >= dur) {
            break;
        }
    }
}

fn main() -> isize {
    let mut T = 5i32;
    while T > 0 {
        sleep(1000);
        println!("{} !", T);
        T -= 1;
    }
    println!("End!");
    233
}

entry!(main);
```

```
[>] sleep
5 !
4 !
3 !
2 !
1 !
End!
Process [3] exited with code: 233
```

可以看到该程序可以正确倒数 `5s`，说明 `sys_time` 系统调用实现基本无误！